

Universidad Peruana de Ciencias Aplicadas



Ciencias de la Computación

1ACC0184 - Complejidad Algorítmica

INFORME DEL TRABAJO FINAL (TB2)

Sección: 17923

Profesor:

Cesar Enrique Salas Arbaiza

Código de alumno:	Nombres y apellidos:
202515871	Alexis Sebastián Martín Farro
202212163	Fernando Sebastián Reque Salas
202414901	Álvaro Cabello García

Lima, Julio 2026

Contenido

1. Descripción del problema	3
2. Descripción del Conjunto de Datos (Dataset)	3
2.1. Origen de los Datos	3
2.2. Motivo del Análisis	4
2.3. Relación con grafos	4
3. Propuesta	5
4. Diseño de aplicativo	6
5. Validación de datos y pruebas	9
5.1. Entradas y salidas del sistema	9
5.2. Interpretación de resultados	9
5.3. Pruebas unitarias	9
6. Conclusiones	10
7. Anexos	10
7.1. Estructura del proyecto	10
7.2. Parámetros por defecto del modelo	11
Referencias	12

1. Descripción del problema

El litoral de Lima Metropolitana y la provincia del Callao enfrentan una severa problemática de contaminación marina, agravada por episodios como el derrame de hidrocarburos de Ventanilla, cuyos residuos persisten en el ecosistema costero (CooperAcción, 2025; Sociedad Peruana de Derecho Ambiental, 2024). El monitoreo continuo de estas zonas exige Vehículos Submarinos Autónomos (AUV) de reconocimiento, capaces de recorrer el medio marino muestreando contaminantes de forma autónoma (Eichhorn, 2009). No obstante, su autonomía está fuertemente condicionada por la capacidad de sus baterías (Kularatne et al., 2018).

El reto central es que el AUV opera en un medio altamente dinámico, donde navegar en línea recta ignora la física del fluido: vencer el arrastre (*drag*) que el agua opone al avance consume energía de forma desproporcionada (Doshi et al., 2023), lo que acorta la misión y arriesga la pérdida del equipo. La distancia más corta no equivale al menor esfuerzo energético, pues una misma corriente favorece o se opone al desplazamiento según la dirección en que se viaje.

Dado que la autonomía no permite recorrer todo el dominio, el sistema concentra el reconocimiento donde el contaminante se acumula: la fuente de emisión y las zonas de convergencia del campo de corrientes, donde el flujo se frena y concentra las partículas. La misión consiste, entonces, en visitar ese conjunto de zonas prioritarias y retornar a la base con el menor gasto neto de energía.

Para resolverlo, el océano se representa como un grafo dirigido, ponderado y asimétrico, donde el peso de cada arista es la energía neta de desplazamiento e incorpora la regeneración de batería: cuando la corriente es suficientemente favorable, el vehículo apaga la propulsión y sus turbinas operan como generador, recuperando una fracción de la energía del flujo relativo. Esta posibilidad se adopta como un supuesto de modelado físicamente motivado por la literatura de cosecha de energía hidrocínética, que ha estudiado sistemas de turbinas submarinas (Tandon et al., 2019) y otros captadores de energía de corrientes oceánicas (Olinger & Wang, 2015). Como esta capacidad introduce transiciones de costo neto negativo, las rutas de mínima energía entre zonas se calculan con el algoritmo de Bellman-Ford, que admite pesos negativos y detecta ciclos negativos; y el orden óptimo de visita de las zonas se resuelve como un problema del viajante (TSP) sobre ese conjunto reducido de puntos. El objetivo es determinar la ruta de reconocimiento de mínimo consumo energético que visite las zonas de interés y regrese a la base.

2. Descripción del Conjunto de Datos (Dataset)

2.1. Origen de los Datos

Los datos provienen del Copernicus Marine Service (CMEMS), del producto GLOBAL_ANALYSISFORECAST_PHY_001_024 (*Global Ocean Physics Analysis and Forecast*), que entrega análisis y pronósticos operativos del estado físico del océano global. La descarga se realizó en formato NetCDF (.nc), un estándar para datos científicos multidimensionales que almacena variables georreferenciadas sobre una malla regular de latitud, longitud, profundidad y tiempo.

Se emplean las dos componentes de la corriente marina: u_o (componente zonal, velocidad hacia el este) y v_o (componente meridional, velocidad hacia el norte), en m/s. El dominio es un recorte sobre el litoral de Lima y Callao, entre las latitudes 13.08° S y 11.42° S y las longitudes 78.42° O y 76.58° O. La malla tiene una resolución de $1/12^\circ$ ($\approx 0.083^\circ$, unos 9 km), discretizada en 21 niveles de latitud $\times$ 23 de longitud $\times$ 4 de profundidad (0.49, 1.54, 2.65 y 3.82 m, capas cercanas a la superficie). La dimensión temporal contiene 10 pasos diarios (cada 24 h), del 13 al 22 de mayo de 2026.

2.2. Motivo del Análisis

Se eligió este conjunto porque las corrientes marinas son el factor físico que gobierna a la vez los dos aspectos centrales del problema. Por un lado, determinan el costo energético del desplazamiento del AUV: moverse a favor o en contra del flujo cambia drásticamente la energía requerida. Por otro, son el mecanismo que transporta y acumula los contaminantes, de modo que definen dónde tiene sentido muestrear.

La magnitud de la corriente en el dominio llega a 0.63 m/s, con un promedio de 0.24 m/s. Estos valores son comparables a la velocidad de crucero típica de un AUV (del orden de 0.5 m/s), lo que confirma que la corriente no es una perturbación menor sino un factor dominante en el balance energético: ignorarla al planificar la ruta tendría un costo elevado. Al tratarse de datos operativos reales —no sintéticos—, el modelo se apoya en condiciones oceanográficas verosímiles del mar de Lima, lo que da realismo y reproducibilidad al análisis. La región se seleccionó por su relevancia ambiental, asociada a episodios como el derrame de Ventanilla.

2.3. Relación con grafos

El dataset se traduce en el grafo de la siguiente forma. Cada celda de la malla con coordenada (latitud, longitud, profundidad) que corresponde a agua navegable se convierte en un nodo; las celdas marcadas como tierra (valores NaN en u_o/v_o) se descartan, actuando como obstáculos y no como nodos. De las 1 932 celdas teóricas ($21 \times 23 \times 4$), 364 son tierra (18.8 %), por lo que el grafo queda con **1 568 nodos navegables**.

Cada nodo se conecta con sus 26 vecinos —las celdas adyacentes en las tres dimensiones—, generando aristas dirigidas. El peso de cada arista no es la distancia, sino la energía neta de desplazamiento, calculada a partir de las corrientes u_o y v_o locales mediante la función de costo de dos regímenes (propulsión y regeneración). Como ir de A a B no cuesta lo mismo que de B a A, el grafo es dirigido y asimétrico.

Finalmente, el propio campo de corrientes define los puntos de interés de la misión. A partir de u_o y v_o se calcula la divergencia del flujo; las zonas de convergencia (divergencia negativa, donde el flujo se frena y concentra las partículas) constituyen los *waypoints* prioritarios del reconocimiento. Para evitar seleccionar celdas contiguas que representen el mismo foco, se aplica un criterio de distancia mínima entre candidatos. Además, se incorporan *centinelas offshore*: celdas en la franja oceánica abierta con la corriente entrante más rápida, orientadas a la detección temprana de derrames que aún no han llegado a las zonas de acumulación. Finalmente, el punto de partida y retorno se fija en el puerto del Callao ($\approx 12.05^\circ$ S, 77.15° O), celda navegable más cercana a esa coordenada en la malla.

3. Propuesta

1. Modelado del problema como grafo

El espacio marino se modela como un grafo dirigido y ponderado $G = (V, E)$. Cada celda navegable de la malla —terna (latitud, longitud, profundidad) con dato de corriente— es un nodo; las celdas de tierra se excluyen. Cada nodo se enlaza con sus hasta 26 vecinos en las tres dimensiones, y cada conexión genera **dos aristas dirigidas** ($A \rightarrow B$ y $B \rightarrow A$) con pesos calculados de forma independiente. El peso de una arista no es la distancia geométrica, sino la **energía neta** que el AUV gasta o recupera al recorrerla, derivada de la corriente local. Como la corriente rompe la simetría, $w(A \rightarrow B) \neq w(B \rightarrow A)$: el grafo es asimétrico, y esa es la propiedad que hace no trivial el problema.

2. Función de costo de las aristas

Para una arista dirigida de A hacia B se define su geometría: el desplazamiento (convertido de grados a metros) da una longitud L y un vector unitario de dirección $\hat{e}$. La corriente local es $v_c = (u_o, v_o, 0)$. El vehículo navega a una velocidad de crucero s (parámetro de diseño), por lo que la **velocidad que debe sostener respecto al agua** es:

$$v_r = s \cdot \hat{e} - v_c$$

El arrastre que el agua opone crece con el cubo de esa velocidad relativa (ley de potencia cúbica para arrastre cuadrático) (Doshi et al., 2023). La proyección de la corriente sobre la dirección de avance, $v_{\parallel} = v_c \cdot \hat{e}$, decide el régimen:

Régimen de propulsión (cuando $v_{\parallel} < s$): El motor aporta empuje y se gasta batería. El peso es positivo:

$$w(A \rightarrow B) = k_p \cdot |v_r|^3 \cdot \left(\frac{L}{s}\right)$$

Régimen de regeneración (cuando $v_{\parallel} \geq s$): El motor se apaga y las turbinas operan como generador, recuperando energía del flujo relativo. El peso es negativo:

$$w(A \rightarrow B) = -k_r \cdot \eta \cdot |v_r|^3 \cdot \left(\frac{L}{s}\right)$$

donde $\eta \in (0, 1)$ es la eficiencia de conversión y k_p, k_r, s son parámetros del modelo que se calibran en la implementación. La energía recuperada se topa en la capacidad de batería $E_{\max}$. La asimetría es automática: al invertir el sentido ($B \rightarrow A$), $\hat{e}$ y $v_{\parallel}$ cambian de signo, y una arista que regenera en un sentido cuesta caro en el opuesto.

3. Identificación de las zonas prioritarias

La misión no recibe las zonas desde afuera: las deriva del propio campo de corrientes. A partir de u_o y v_o sobre la malla se calcula la **divergencia horizontal** del flujo por diferencias finitas:

$$\text{div}(x, y) = \frac{\partial u_o}{\partial x} + \frac{\partial v_o}{\partial y}$$

Una divergencia **negativa** indica que el flujo converge. Las celdas con divergencia más negativa son las candidatas primarias a zona de muestreo; no obstante, aplicar este criterio directamente produce agrupamientos de celdas contiguas que representan la misma zona física. Para evitarlo se impone una **distancia mínima entre waypoints** seleccionados, asegurando cobertura espacial distribuida.

A las zonas de convergencia se suman **centinelas offshore**: celdas en la franja oceánica abierta (40 % más occidental del dominio) con la componente zonal de corriente más positiva ($u_o > 0$, flujo entrante hacia la costa). Estos centinelas permiten detectar derrames antes de que el flujo los transporte hasta las zonas de acumulación. El punto de base —partida y retorno obligatorio del AUV— se fija en el puerto del Callao, tomando la celda navegable más cercana a esa coordenada real.

El conjunto de *waypoints* queda integrado por: k_c zonas de convergencia deduplicadas, k_s centinelas offshore y la base, con $k_c + k_s$ pequeño (del orden de 7 a 9) para mantener la fase ATSP tratable.

4. Algoritmo de solución

Como el régimen de regeneración produce aristas de costo negativo, el algoritmo de Dijkstra queda descartado. Se emplea **Bellman-Ford**, que relaja las E aristas $V - 1$ veces y admite pesos negativos. Una pasada adicional verifica la ausencia de ciclos negativos. El modelo se calibra para que no aparezcan: como la cosecha es pasiva y recupera solo una fracción ($\eta < 1$, con $k_r \eta$ menor que el costo de propulsión k_p), ningún recorrido cerrado debería rendir energía neta, de modo que el costo de mínima energía está bien definido. La detección de Bellman-Ford funciona así como control de validación del modelo —un ciclo negativo revelaría una mala calibración de los parámetros—; y, en todo caso, la energía recuperada a lo largo de la ruta se topa siempre en la capacidad de batería $E_{\max}$.

La misión de reconocimiento se resuelve en dos capas, siguiendo el esquema de planificación jerárquica propuesto para robots marinos en campos de flujo (Lee et al., 2020). Primero, con Bellman-Ford se calcula la ruta de mínima energía entre cada par de zonas, obteniendo una matriz de costos. Segundo, sobre esa matriz se determina el orden óptimo de visita: un Problema del Viajante Asimétrico (ATSP) resuelto de forma exacta por enumeración (fuerza bruta) de los órdenes posibles.

5. Análisis de complejidad

Bellman-Ford tiene complejidad $O(V \cdot E)$. Con $V = 1568$ nodos y hasta 26 vecinos por nodo, $E \approx 40000$ aristas dirigidas, lo que da del orden de 6×10^7 relajaciones —resoluble en segundos. Para k zonas se ejecuta k veces, manteniendo la fase en tiempo polinomial.

La capa de orden (ATSP) es NP-difícil: su resolución exacta crece como $O((k - 1)!)$. Al mantener k pequeño ($k = 7$, solo 720 órdenes), se mantiene el problema tratable.

4. Diseño de aplicativo

Requisitos funcionales:

ID	Descripción del requisito
----	---------------------------

RF-01	El sistema debe cargar un archivo NetCDF (.nc) de Copernicus Marine y extraer las componentes de corriente u_o y v_o sobre la malla de latitud, longitud y profundidad, descartando las celdas de tierra (valores NaN).
RF-02	El sistema debe construir un grafo dirigido y ponderado, conectando cada celda navegable con sus hasta 26 vecinos y asignando a cada arista un peso de energía neta según los regímenes de propulsión y regeneración.
RF-03	El sistema debe calcular la divergencia horizontal del campo de corrientes y seleccionar las k_c zonas de mayor convergencia como <i>waypoints</i> , aplicando un criterio de distancia mínima entre candidatos para evitar redundancia espacial. Debe además incorporar k_s centinelas offshore (celdas con corriente entrante más rápida en la franja oceánica abierta) para detección temprana de derrames, y fijar la base de misión en el puerto del Callao como punto de partida y retorno obligatorio.
RF-04	El sistema debe calcular, mediante el algoritmo de Bellman-Ford, la ruta de mínima energía entre cada par de <i>waypoints</i> y construir la matriz de costos asimétrica.
RF-05	El sistema debe determinar el orden óptimo de visita (ATSP) por enumeración exacta y ensamblar la ruta completa que parte de la base, visita todas las zonas y retorna a ella.
RF-06	El sistema debe permitir al usuario configurar los parámetros del modelo: velocidad de crucero, coeficientes de propulsión y de regeneración, eficiencia de conversión, capacidad de batería y número de zonas a visitar.
RF-07	El sistema debe visualizar gráficamente el dominio: el campo de corrientes, las zonas de convergencia y la ruta óptima resultante sobre el área de estudio.
RF-08	El sistema debe reportar los resultados numéricos de la misión: energía total consumida, costo de cada tramo, orden de visita y aviso ante la detección de ciclos negativos.
RF-09	El sistema debe permitir exportar la ruta y las métricas resultantes a un archivo (CSV y/o imagen).

Requisitos no funcionales:

ID	Descripción del requisito
RNF-01	El cálculo completo de la ruta (construcción del grafo, ejecuciones de Bellman-Ford y fase ATSP) no debe exceder los 60 segundos en un equipo de gama media; cada ejecución de Bellman-Ford sobre el grafo (del orden de 1568 nodos y 40 000 aristas) debe resolverse en pocos segundos.
RNF-02	El núcleo algorítmico debe implementarse en Python; la interfaz gráfica se desarrollará en Streamlit, mostrando el mapa del dominio, el campo de corrientes y la ruta sobre el área de estudio.
RNF-03	La interfaz debe permitir lanzar el cálculo y consultar la ruta sin requerir conocimientos de programación por parte del usuario.

RNF-04	El sistema debe ejecutarse en Windows y Linux empleando librerías estándar del ecosistema científico de Python (numpy, xarray/netCDF4, matplotlib).
RNF-05	El código debe ser modular, separando la carga de datos, la construcción del grafo, los algoritmos y la visualización, de modo que cada componente pueda probarse y mantenerse de forma independiente.
RNF-06	Los resultados deben ser reproducibles y deterministas para un mismo conjunto de datos y de parámetros.
RNF-07	El sistema debe validar las entradas, manejar las celdas inválidas (NaN) y advertir ante la presencia de ciclos negativos sin interrumpir la ejecución.

Diseño de Interfaz de usuario:

La interfaz es una aplicación web desarrollada con Streamlit, accesible desde el navegador con el comando `streamlit run app.py`. Se organiza en dos zonas:

Barra lateral (parámetros y dataset). El usuario configura el modelo mediante controles interactivos: velocidad de crucero s [m/s], eficiencia de regeneración η , coeficientes k_p y k_r , número de zonas de convergencia k_c , número de centinelas offshore k_s , separación mínima entre waypoints [celdas], capacidad máxima de batería $E_{\max}$ [J] y porcentaje de carga inicial. El dataset se carga subiendo un archivo NetCDF o usando el archivo incluido por defecto (lima3.nc).

Área principal. Al presionar el botón *Calcular ruta óptima*, el sistema ejecuta el pipeline completo y presenta los resultados en cuatro pestañas:

- **Zonas y divergencia:** mapa del campo de divergencia sobre el dominio de Lima, con las zonas de convergencia ($C1 \dots k_c$), los centinelas offshore ($S1 \dots k_s$) y la base del Callao señalados.
- **Ruta 2D:** ruta completa del AUV proyectada en lat-lon y coloreada según la profundidad de cada segmento.
- **Ruta 3D por capas:** los cuatro niveles de profundidad se apilan como planos horizontales coloreados por rapidez de corriente; la ruta se traza en rojo entre ellos.
- **Batería:** evolución de la carga a lo largo de la misión (eje X: distancia recorrida [km]), con sombreado de tramos de propulsión (naranja) y regeneración (verde), y banda roja de peligro por debajo del 20 % de $E_{\max}$.

Sobre el área principal también se muestran cuatro métricas numéricas (energía total, consumo de propulsión, energía regenerada y nivel mínimo de batería), una tabla desglosada por tramo, y un botón de exportación de la ruta en CSV. Las figuras se generan ejecutando `python -m src.visualizacion` desde la raíz del proyecto y se guardan en `outputs/figuras/`.

5. Validación de datos y pruebas

5.1. Entradas y salidas del sistema

Entradas: un archivo NetCDF de Copernicus Marine con las variables u_0 y v_0 sobre una malla de latitud $\times$ longitud $\times$ profundidad, y los parámetros del modelo: s , η , k_p , k_r , k_c , k_s , $E_{\max}$ y separación mínima entre waypoints.

Salidas:

- Ruta óptima como secuencia de nodos (`prof_idx`, `lat_idx`, `lon_idx`).
- Métricas de la misión: energía total [J], energía de propulsión, energía regenerada y nivel mínimo de batería.
- Archivo CSV exportable con columnas `paso`, `prof_m`, `lat_deg`, `lon_deg`.
- Cuatro visualizaciones descargables (zonas y divergencia, ruta 2D, ruta 3D por capas, estado de batería).

5.2. Interpretación de resultados

La energía total de la misión es la suma de los pesos de las aristas del camino óptimo. Los valores negativos en aristas individuales indican tramos de regeneración; el valor total puede ser positivo o negativo según el balance global propulsión-regeneración. El nivel mínimo de batería determina la viabilidad de la misión: si desciende a 0 J, el AUV se quedaría sin energía antes de regresar a la base. Se recomienda mantener ese mínimo por encima del 20 % de $E_{\max}$ (zona segura marcada en rojo en el gráfico de batería).

La detección de ciclos negativos funciona como control de consistencia del modelo: si $k_r \cdot \eta \geq k_p$, el AUV ganaría energía en recorridos circulares, lo que violaría la conservación de energía. En ese caso el sistema emite una advertencia y los resultados pueden no ser fiables.

5.3. Pruebas unitarias

Las pruebas del módulo `src/algoritmos.py` están en `tests/test_algoritmos.py` y se ejecutan con `pytest` desde la raíz del proyecto:

- **test_bellman_ford_camino_simple:** en un grafo lineal $A \rightarrow B \rightarrow C$ con pesos 3 y 5, verifica que la distancia calculada a C es 8, y que los predecesores son correctos.
- **test_bellman_ford_prefiere_arista_negativa:** con dos caminos alternativos hacia el mismo destino —uno directo de costo 10 y otro con arista de regeneración de costo neto 2— verifica que Bellman-Ford elige el camino de menor energía.
- **test_detecta_ciclo_negativo:** construye un ciclo $A \rightarrow B \rightarrow C \rightarrow A$ con peso total -2 J y verifica que `hay_ciclo_negativo` lo detecta correctamente.
- **test_atsp_orden_optimo_caso_pequeno:** para una matriz de costos 3×3 con solución óptima conocida (orden [0, 1, 2, 0], costo 3 J), verifica que `atsp_fuerza_bruta` devuelve exactamente ese resultado.

6. Conclusiones

1. **Bellman-Ford es la elección correcta cuando existen aristas de peso negativo.** La posibilidad de regeneración de batería introduce tramos de costo negativo en el grafo, lo que descarta a Dijkstra (no garantiza corrección en esas condiciones). Además del cálculo de rutas mínimas, Bellman-Ford proporciona de forma natural la detección de ciclos negativos en una pasada adicional de relajación. Esto actúa como mecanismo de validación del modelo: un ciclo de energía neta negativa revela una calibración defectuosa de los parámetros ($k_r \cdot \eta \geq k_p$) que haría que el AUV «ganase» energía dando vueltas, violando la conservación de energía. La terminación anticipada (detener en cuanto no se relaja ninguna arista) reduce el número de iteraciones de forma significativa sobre mallas regulares, logrando convergencia en decenas de pasadas en lugar de $V - 1$.
2. **La factorización en dos capas hace tratable un problema de otra manera intratable.** Intentar resolver el reconocimiento completo como un TSP sobre los 1 568 nodos del grafo sería inviable por enumeración. La estrategia jerárquica reduce primero el espacio a una matriz de costos entre los k waypoints ejecutando Bellman-Ford k veces (en tiempo polinomial $O(k \cdot V \cdot E)$), y luego resuelve un ATSP de tamaño k por fuerza bruta en $O((k - 1)!)$. Manteniendo $k \leq 9$ —lo que produce como mucho 40 320 órdenes a evaluar— se obtiene la solución exacta en segundos, sin sacrificar optimalidad.
3. **Las corrientes marinas son un factor dominante en el balance energético, no una perturbación menor.** Con velocidades de hasta 0.63 m/s en el dominio de Lima y una velocidad de cruce del AUV de 0.5 m/s, la corriente puede superar la velocidad del vehículo y cambiar el régimen de propulsión a regeneración. El mismo tramo geográfico puede tener costos radicalmente distintos según la dirección recorrida, lo que hace que el grafo sea asimétrico: $w(A \rightarrow B) \neq w(B \rightarrow A)$. Planificar la ruta por distancia mínima en lugar de energía mínima llevaría a rutas subóptimas o, en casos extremos, inviables por agotamiento de batería.
4. **Líneas de trabajo futuro.** Varias extensiones naturales se abren a partir de este trabajo: (a) reemplazar la enumeración exacta del ATSP por heurísticas eficientes (2-opt, Lin-Kernighan) para escalar a valores de k mayores sin sacrificar calidad de solución; (b) incorporar la dimensión temporal del dataset —el producto CMEMS incluye múltiples pasos diarios— para optimizar la hora de inicio y el orden de visita considerando corrientes variables a lo largo de la misión; (c) modelar restricciones adicionales como tiempos de permanencia en cada waypoint para muestreo, recargas en superficie mediante paneles solares, o zonas de paso prohibido.

7. Anexos

7.1. Estructura del proyecto

Archivo / directorio	Descripción
----------------------	-------------

data/lima3.nc	Dataset NetCDF de Copernicus Marine (producto GLOBAL_ANALYSISFORECAST_PHY_001_024), recorte litoral Lima–Callao, mayo 2026.
src/datos.py	Carga y preprocesamiento del NetCDF; estructura CampoCorrientes (RF-01).
src/config.py	Parámetros inmutables del modelo (ParametrosModelo).
src/grafos.py	Clase Grafo y función de costo de aristas por régimen (RF-02, RF-03).
src/zonas.py	Cálculo de divergencia y selección de waypoints y centinelas (RF-03).
src/algoritmos.py	Bellman-Ford, detección de ciclos negativos y ATSP por fuerza bruta (RF-04, RF-05).
src/metricas.py	Resumen de misión, simulación de batería y exportación CSV (RF-08, RF-09).
src/visualizacion.py	Visualizaciones matplotlib: zonas, ruta 2D/3D, batería (RF-07).
app.py	Interfaz web Streamlit; orquesta los módulos anteriores (RF-06 a RF-09).
tests/ test_algoritmos.py	Pruebas unitarias de Bellman-Ford y ATSP (ejecutar con pytest).
outputs/figuras/	Figuras generadas al ejecutar <code>python -m src.visualizacion</code> .
outputs/rutas/	Archivos CSV de rutas exportadas desde la interfaz.

7.2. Parámetros por defecto del modelo

Parámetro	Valor por defecto	Descripción
s	0.5 m/s	Velocidad de crucero del AUV respecto al agua.
k_p	1.0	Coeficiente de costo en régimen de propulsión.
k_r	1.0	Coeficiente de recuperación en régimen de regeneración.
η	0.30	Eficiencia de conversión de la regeneración.
$E_{\max}$	1 000 000 J	Capacidad máxima de la batería.
k_c	6	Número de zonas de convergencia a visitar.
k_s	2	Número de centinelas offshore.

Referencias

- CooperAcción. (2025,). *¿Repsol limpió el desastre? Situación actual y retos pendientes para la recuperación de la vida marina a tres años del derrame de petróleo*. <https://cooperaccion.org.pe/wp-content/uploads/2025/01/repsol-1.pdf>
- Doshi, M. M., Bhabra, M. S., & Lermusiaux, P. F. J. (2023). Energy-time optimal path planning in dynamic flows: Theory and schemes. *Computer Methods in Applied Mechanics and Engineering*, 405, 115865. <https://doi.org/10.1016/j.cma.2022.115865>
- Eichhorn, M. (2009). Optimal path planning for AUVs in time-varying ocean flows. *OCEANS 2009-EUROPE*, 1-6. <https://doi.org/10.13140/2.1.2662.4962>
- Kularatne, D., Hajieghrary, H., & Hsieh, M. A. (2018). Optimal path planning in time-varying flows with forecasting uncertainties. *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 4857-4864. <https://doi.org/10.1109/ICRA.2018.8460221>
- Lee, J. J. H., Yoo, C., Anstee, S., & Fitch, R. (2020,). Hierarchical planning in time-dependent flow fields for marine robots. *2020 IEEE International Conference on Robotics and Automation (ICRA)*. <http://hdl.handle.net/10453/148185>
- Olinger, D. J., & Wang, Y. (2015). Hydrokinetic energy harvesting using tethered undersea kites. *Journal of Renewable and Sustainable Energy*, 7(4). <https://doi.org/10.1063/1.4926769>
- Sociedad Peruana de Derecho Ambiental. (2024,). *A un mes del derrame de petróleo*. <https://spda.org.pe/wp-content/uploads/2024/01/A-un-mes-del-derrame-de-petroleo.pdf>
- Tandon, S., Divi, S., Muglia, M., Vermillion, C., & Mazzoleni, A. (2019). Modeling and dynamic analysis of a mobile underwater turbine system for harvesting Marine Hydrokinetic Energy. *Ocean Engineering*, 187. <https://doi.org/10.1016/j.oceaneng.2019.05.051>